

OccuEVRoute Report / PPT 协作写作指南

本文档用于多人协作完成 OccuEVRoute 的 report 和 presentation。当前目标是 **10 分钟 presentation** 和 **约 1500 英文单词 report**。建议大家先统一阅读本指南，再分别负责自己的章节，避免内容重复、口径不一致，或者只讲代码而没有讲清楚项目价值。

总体叙事线

整份 report / PPT 应围绕一个核心问题展开：

在深圳城市路网中，给定 EV 用户位置、电量和偏好约束，系统如何推荐一个既能到达、又尽量快、且更可能有空位的充电站？

推荐的逻辑顺序是：

- 城市 EV 充电痛点

→ 问题形式化

→ 数据来源与处理

→ 路网搜索与约束过滤

→ 占用率机器学习预测

→ 搜索结果、ML 结果、复杂度与局限

写作时要把项目讲成一个完整系统，而不是六个互不相关的模块。Search Strategy 解释“怎么到达”，Machine Learning 解释“到达时可能有多拥挤”，Performance 汇总 search results、ML results、complexity 和 limitations。

因为 presentation 只有 10 分钟，report 只有约 1500 英文单词，所有章节都应服务主线，不要平均铺开。建议时间和篇幅分配：

部分	PPT 时间	Report 篇幅
Introduction	1 min	150-200 words
Problem Definition	1.5 min	220-260 words
Dataset	1.5 min	250-300 words
Search Strategy	2 min	300-350 words
Machine Learning	2 min	300-350 words
Performance	2 min	250-300 words

Search Strategy 和 Machine Learning 是技术核心，可以多讲；Introduction 和 Problem Definition 要短而清楚。

建议 PPT 控制在 9-11 页：

页码	内容
1	Title + one-sentence project goal
2	Motivation / Introduction
3	Problem Definition
4	Dataset overview
5	Route planning workflow
6	Search Strategy: BFS → Bi-BFS → CH
7	Search Strategy: UCS → A* → ALT A*
8	Machine Learning model
9	ML results / feature importance
10	Performance: search results + ML results + complexity
11	Limitations / conclusion, optional if time allows

1500-word report 建议只保留 6 个一级小节，每节 1-3 段。不要把代码片段大段放进 report；代码路径可以放在句子里或附录里。

已有项目资料优先级：

资料	用途
README.md	项目概览、运行方式、数据与模块结构
PRODUCT.md	产品目标、用户、演示场景
docs/presentation_notes_cn.md	六个模块的中文讲解稿和代码线索
docs/models/occupancy_horizon_model.md	机器学习模型、特征、结果
docs/figures/	模型指标、SHAP、特征重要性等图表
src/route_planning/	搜索、约束、推荐逻辑
src/waiting_prediction/	训练、特征构造、模型评估
src/data_processing/	数据处理、路网、POI、landmark 预处理

章节 1：Introduction

这一节要回答什么

Introduction 要让听众在 1-2 分钟内明白项目是什么、为什么值得做、最终系统能展示什么。

建议写：

- 项目一句话定义：OccuEVRoute 是一个面向深圳的 EV 充电路线规划与充电站推荐系统。
- 用户输入：当前位置、SOC / 电池容量、最大行驶时间、搜索半径、最低到达电量等。

- 系统输出：推荐充电站排名、路线、行驶时间、距离、到达 SOC、预测占用率、诊断原因。
- 项目集成了三类能力：路网搜索、CSP 可行性约束、XGBoost 占用率预测。
- 课程 demo 价值：不仅给出结果，还能解释为什么这个站被推荐、为什么其他站被过滤。

建议 PPT 图：

- 一张系统流程图：User Input → Candidate Stations → Constraint Filtering → Search Algorithm → ML Occupancy Prediction → Ranked Recommendations。
- 一张前端地图截图，突出 map、输入面板、推荐列表和路线。

避免：

- 不要一开始讲太多代码细节。
- 不要把它写成普通导航软件；重点是“充电约束 + 占用率预测 + 可解释推荐”。

章节 2：Problem Definition

这一节要回答什么

Problem Definition 要把真实问题转成可计算问题，让后面的算法选择显得自然。

建议写：

- EV charging route planning 的核心困难：
 - 最近或最短距离的站不一定是行驶时间最优的站。
 - 距离近的站不一定有更低的拥挤风险。
 - 城市路网搜索规模大，需要有效算法。
- 输入变量：
 - 用户位置：latitude / longitude。
 - 车辆状态：current SOC、battery capacity、energy consumption。
 - 用户约束：max search radius、max drive time、minimum arrival SOC、minimum charger count。
- 输出目标：
 - 找到满足约束的候选站。
 - 对每个站计算实际路网路径、行驶时间、距离和到达电量。
 - 结合预测占用率辅助判断等待风险。
- 约束设计：
 - Pre-check (constraint search)：搜索前使用静态约束筛选候选站，例如直线距离、充电桩数量、道路接入距离。它的作用是缩小搜索空间，避免对明显不可行的站点运行昂贵的路网搜索。
 - Post-check (constraint validation)：搜索后基于真实路径结果验证行驶时间、电量消耗、到达 SOC。它的作用是保证最终推荐结果不仅“搜得到”，而且满足 EV 可达性和用户约束。
 - Ranking (multi-key sorting)：对通过 post-check 的可行站点排序。系统先合并 ML 预测占用率，再按用户选择的 ranking metric 做多字段排序；其中 occupancy 只作为拥挤风险信号，不表示等待时间。
- CSP 在本项目中的位置：
 - CSP 不需要单独开一个大章节，建议放在 Problem Definition 里讲“什么叫可行解”，再在 Search Strategy 里讲它如何嵌入路线搜索。
- 简单讲法是：pre-check 负责约束搜索空间，post-check 负责约束验证，ranking 负责把可行站点按用户当前目标排出优先级。

- 排序目标：
 - 默认 balanced: $ml_rank_score = drive_time_min / max_drive_time_min + predicted_occupancy_rate$ ，分数越低越靠前。这里先用用户设置的最大行驶时间归一化 drive time，再把 occupancy 作为拥挤风险项加入排序。
 - 可切换为 shortest drive time、shortest distance、lowest predicted occupancy、highest arrival SOC。
 - 实现上后端调用 pandas `sort_values` 做多字段排序；汇报重点讲排序键和业务含义即可。

建议 PPT 图：

- 一页“问题形式化”表格：Input / Constraints / Objective / Output。
- 一页“CSP-guided recommendation”图：pre-check → route search → post-check → ranking。

可引用代码：

- `src/route_planning/constraints.py`
- `src/route_planning/recommender.py`

避免：

- 不要只列公式而不解释业务含义。
- 不要把 ML 预测也写成硬约束；当前更合理的表达是“辅助排序和等待风险解释”。

章节 3：Dataset

这一节要回答什么

Dataset 要说明系统依赖哪些数据、数据如何处理、为什么这些数据足以支持路线推荐和占用率预测。

建议分成两类数据讲：

路线规划数据

需要写：

- 深圳可行驶路网：由 OSMnx 下载并处理成 GraphML。
- 充电站道路接入：将站点投影或连接到附近道路节点 / 边。
- Landmark 距离表：用于 A* / ALT 启发式加速。
- CH index：用于 CH 双端 Dijkstra 的离线预处理索引，包含节点 rank、upward / reverse upward edges，以及 shortcut edges。
- POI 特征：统计站点周边生活服务、商业住宅、餐饮等环境信息。
- 深圳边界：用于裁剪路网和过滤站点。

可引用路径：

- `data/processed/shenzhen_drive_with_station_access.graphml`
- `data/processed/station_road_access.csv`
- `data/processed/landmark_distances.npz`
- `data/processed/ch_index.pkl`
- `data/processed/station_poi_features.csv`
- `data/processed/shenzhen_boundary.geojson`

这里不用展开 CH index 的内部结构；Dataset 只说明它是由路网图预处理得到的派生索引，用于支持后文的 CH 双端 Dijkstra 即可。

占用率预测数据

需要写：

- 数据来源：UrbanEV 深圳公开数据集。
- 时间范围：2022-09-01 至 2023-02-28。
- 当前模型覆盖：1423 个站点。
- 训练样本设计：每行表示某站点某时刻，标签是未来 horizon 的 `target_occupancy_rate`。
- Horizon：5, 10, 15, 20, 30, 45, 60, 90, 120 分钟。
- 总样本数：720000；训练样本 576236；测试样本 143764。
- 时间切分：训练集到 2023-01-23，测试集为 2023-01-23 之后到 2023-02-28。

建议讲清楚的数据处理原则：

- 时间切分而不是随机切分，更接近真实未来预测。
- Lag / rolling 特征只使用当前时刻之前的数据。
- 历史画像和邻居画像只在训练集统计，再映射到测试集，避免数据泄漏。
- POI 和站点静态特征解释空间上下文，天气和时间特征解释需求波动。

建议 PPT 图：

- 数据来源表。
- 特征类别表：time / weather / station / POI / historical profile / neighbor / lag。
- 如果空间允许，放一张站点或 POI 分布图。

可引用文档：

- `docs/models/occupancy_horizon_model.md`
- `docs/figures/occupancy_horizon_feature_importance.csv`
- `docs/figures/occupancy_horizon_shap_bar.png`
- `docs/figures/occupancy_horizon_shap_summary.png`

避免：

- 不要只写“we used UrbanEV dataset”，必须说明数据粒度、标签构造、切分方式。
- 不要把未来真实占用率说成输入特征。

章节 4：Search Strategy

这一节要回答什么

Search Strategy 要解释系统如何在真实路网上找到候选站路径，以及为什么要比较两组搜索思想：

- 第一组：BFS → Bidirectional BFS → CH Bidirectional Dijkstra，主线是“从单向扩展到双向扩展，再到预处理后的双向最短路查询”。
- 第二组：UCS → A* → ALT A*，主线是“从带权最短路到启发式搜索，再到 landmark 启发式优化”。

建议写：

- 图模型定义：
 - 节点表示道路交叉点或路网节点。
 - 边表示可行驶道路。
 - 边权重主要使用 travel time。
- 候选站流程：
 - 先选半径内候选充电站。
 - 将用户位置和站点连接 / snap 到路网。
 - 搜索前运行 pre-check (constraint search)，先过滤明显不可行站点，避免对每个站都做昂贵路网搜索。
 - 对每个候选站运行搜索算法。
 - 搜索结果进入 post-check (constraint validation)，用真实路径检查 drive time、energy consumption 和 arrival SOC。
 - 最后做 ranking (multi-key sorting)：默认按 $ml_rank_score = drive_time_min / max_drive_time_min + predicted_occupancy_rate$ 排序，也可切换为 drive time、distance、predicted occupancy 或 arrival SOC。occupancy 在这里是拥挤风险信号，不是等待时间。
- BFS：
 - 按层扩展，适合解释基础图搜索。
 - 不考虑边权，不能保证真实行驶时间最短。
- Bidirectional BFS：
 - 从起点和终点两端扩展，用 meeting node 拼接路径。
 - 相比单向 BFS，它展示了如何通过双端 frontier 缩小搜索空间。
 - 但它仍然不考虑边权，所以不能保证 travel-time shortest path。
- CH Bidirectional Dijkstra：
 - CH 指 Contraction Hierarchy，是一种“先离线预处理、再快速查询”的最短路加速方法。
 - 预处理阶段按节点重要性逐步 contract 道路节点，并加入 shortcut edge，保存到 `data/processed/ch_index.pkl`。
 - 查询阶段只沿 rank 更高方向的 upward graph 搜索；从起点正向、从终点反向同时运行 Dijkstra。
 - 两个 frontier 在 meeting node 相遇后，用 CH index unpack shortcut，还原成原始道路节点路径。
 - 它的路径成本仍以 travel time 为权重，目标是得到与 UCS 一致的最短时间结果，但通过预处理减少在线查询开销。
 - 在讲法上，它应接在 Bidirectional BFS 后面：先讲“双端搜索”思想，再讲“CH 如何让双端 Dijkstra 在大路网上更快”。
- UCS：
 - 按累计行驶时间扩展。
 - 可以保证最短时间路径，但搜索范围可能较大。
- A*：
 - 在 UCS 上加入启发式。
 - $f(n) = g(n) + h(n)$ 。
 - 普通 A* 使用直线距离估计，通常能减少无关方向上的扩展。
- ALT A*：
 - 使用预计算 landmark 距离表，通过三角不等式得到更强的启发式。
 - 在讲法上，它应接在 A* 后面：先讲“启发式搜索”，再讲“landmark 如何让启发式更贴近真实路网距离”。

建议在 report 中把搜索算法写成两个小段，不要列成六个散点：

1. **Search-space reduction family:** BFS establishes the baseline, Bidirectional BFS reduces expansion from both ends, and CH Bidirectional Dijkstra combines bidirectional search with offline contraction and shortcuts.
2. **Weighted shortest-path family:** UCS gives the exact travel-time baseline, A* adds a straight-line heuristic, and ALT A* improves the heuristic using precomputed landmark distances.

建议 PPT 图：

- 两组算法对比表：
 - BFS → Bidirectional BFS → CH Bidirectional Dijkstra：single-frontier / two-frontier / preprocessed two-frontier。
 - UCS → A* → ALT A*：no heuristic / straight-line heuristic / landmark heuristic。
- 一张路线搜索流程图：candidate → graph search → route metrics。
- 一张 CH 流程图：offline contraction + shortcut generation → online forward/backward Dijkstra → meeting node → shortcut unpacking。
- 一张算法对比柱状图：expanded nodes / runtime / path time，如已有实验结果可放。

可引用代码：

- `src/route_planning/search_algorithms.py`
- `src/route_planning/ch_search.py`
- `src/route_planning/ch_index.py`
- `src/route_planning/recommender.py`
- `src/route_planning/landmark_heuristic.py`
- `src/route_planning/ch_preprocess.py`
- `tests/test_search_traces.py`

避免：

- 不要说 BFS 找最短路径时不加以限定；应明确它只保证无权图或边数意义上的最短。
- 不要把 A* 说成“总是更快”，应说在启发式有效时通常减少扩展。
- 不要把 CH 说成一种新的目标函数；它仍然是在 travel-time 权重上做最短路，只是把大量计算移到离线预处理。
- 不要忘记说明 shortcut unpacking，否则听众会以为 CH 返回的是抽象 shortcut，不是真实路线。

章节 5：Machine Learning

这一节要回答什么

Machine Learning 要说明为什么需要预测占用率、模型预测什么、特征如何构造、结果是否可信。

建议写：

- 预测目标：
 - 给定当前时刻 t 和 horizon，预测 $t + \text{horizon}$ 的充电站占用率。
 - $\text{occupancy_rate} = \text{busy} / (\text{busy} + \text{idle})$ 。
- 模型：
 - 使用 `XGBRegressor`。
 - 不是为每个 horizon 训练一个模型，而是把 `prediction_horizon_min` 作为输入，训练单一 multi-horizon 模型。

- 特征类别：
 - Horizon features: `prediction_horizon_min`, `horizon_sqrt`。
 - Current / target time features: weekday、hour sin/cos、holiday、peak。
 - Weather: temperature、humidity、rain。
 - Station static: longitude、latitude、charge count、TAZID。
 - Price: service price、电价。
 - POI context: 生活服务、商业住宅、餐饮等。
 - Historical profile: station average occupancy、same-hour profile。
 - Neighbor profile: 附近站点平均占用、距离、充电桩数量等。
 - Lag / rolling: 过去 5 到 60 分钟的占用率趋势。
- 无数据泄漏：
 - 标签是未来值。
 - 输入只使用当前和过去。
 - 训练集统计画像，再应用到测试集。
- 与系统结合：
 - 路网搜索告诉用户能否到达和多久到达。
 - ML 模型估计到达时刻附近的占用风险。
 - 推荐结果可以同时显示 drive time、arrival SOC、predicted occupancy。

当前结果可写：

Metric	Result
Overall R2	0.948742
Overall MAE	0.024937
5 min MAE / R2	0.0138 / 0.9790
120 min MAE / R2	0.0436 / 0.8894

建议 PPT 图：

- 模型输入输出图。
- Horizon performance 表或折线图。
- SHAP summary / feature importance 图。

可引用文档与结果：

- `docs/models/occupancy_horizon_model.md`
- `models/occupancy_horizon_xgboost.pkl`
- `models/occupancy_horizon_features.json`
- `docs/figures/occupancy_horizon_by_horizon_metrics.csv`
- `docs/figures/occupancy_horizon_shap_bar.png`
- `docs/figures/occupancy_horizon_shap_summary.png`

避免：

- 不要说模型直接预测“waiting time”，除非后续实现已经改成等待时间标签。当前文档里的主模型是预测 occupancy rate。
- 不要只报 R2，要解释 MAE 的实际含义：平均占用率误差约 2.5 个百分点。

章节 6: Performance

这一节要回答什么

Performance 要把前面的方法落到结果和取舍上, 说明搜索是否有效、ML 预测是否可信、算法复杂度如何影响设计, 以及当前方案还有哪些限制。

建议分四类写:

Search Results

需要写:

- 比较 BFS、Bidirectional BFS、CH 双端 Dijkstra、UCS、A*、ALT A* 的 runtime、expanded nodes、path quality。
- Performance 的讲法也建议保持两组逻辑:
 - BFS → Bidirectional BFS → CH: 展示搜索空间如何从单向扩展、双向扩展, 进一步到预处理后的双向最短路查询。
 - UCS → A* → ALT A*: 展示 travel-time 最短路如何从无启发式扩展到 straight-line heuristic, 再到 landmark heuristic。
- 强调 BFS / Bidirectional BFS 主要作为 baseline 和可视化教学算法; UCS 是无启发式的 travel-time 最优搜索; A* / ALT A* 用启发式减少扩展; CH 双端 Dijkstra 用离线预处理加速在线最短路查询。
- CH 的性能说明应分清两个阶段:
 - Offline preprocessing: `src/route_planning/ch_preprocess.py` 生成 `data/processed/ch_index.pkl`, 包含节点 rank、upward edge、reverse upward edge 和 shortcut。
 - Online query: `src/route_planning/ch_search.py` 从起点和终点两侧运行 Dijkstra, 在 frontier 不能改进当前最优解时停止, 并还原 shortcut 路径。
- 如果放实验表, 建议把 CH 的 preprocessing time 和 query time 分开写; 不要把一次性预处理成本混进单次推荐查询时间。

建议指标:

- 单次查询平均耗时。
- 候选站数量。
- 成功找到路径比例。
- 每种算法扩展节点数。
- CH shortcut 数量、index size、preprocessing time。
- 返回 top-k 推荐的端到端耗时。

ML Results

需要写:

- Overall MAE / R2。
- 按 horizon 展示误差随预测时间变长而上升。
- SHAP 或 feature importance 解释模型主要依赖哪些特征。
- 说明短期预测更准, 长期预测仍可用于趋势判断。

建议引用:

- Overall R2 = 0.948742。
- Overall MAE = 0.024937。
- 5 min R2 = 0.9790。
- 120 min R2 = 0.8894。

Complexity

需要写：

- BFS / Bidirectional BFS 作为教学 baseline，说明无权搜索在真实 travel-time 权重下的局限。
- UCS 能保证 travel-time 最短路，但在大路网上扩展范围较大。
- A* / ALT A* 通过启发式减少扩展；ALT 的 landmark 距离表把一部分计算转移到预处理。
- CH 双端 Dijkstra 把大量最短路加速工作放到离线 preprocessing，在线查询更快，但需要额外 index 和 shortcut unpacking。
- 复杂度分析不用写成纯公式堆砌，要和“为什么选择这些算法、为什么要预处理”连起来。

Limitations

需要写：

- Occupancy prediction 预测的是 occupancy rate，不是 waiting time。
- Balanced ranking 中 occupancy 只是拥挤风险信号，权重是 heuristic。
- 搜索和推荐基于静态路网及当前数据快照，没有实时交通、实时排队或充电功率变化。
- 当前系统是 course-demo system，重点是算法组合、可解释推荐和结果展示，不承诺生产级导航能力。

Report 与 PPT 的关系

Report 可以更详细，PPT 要更像讲故事：

部分	Report 写法	PPT 写法
Introduction	背景、动机、系统概览	一句话项目 + 流程图 + demo 截图
Problem Definition	输入、约束、目标、公式化	问题表格 + 两阶段过滤
Dataset	来源、规模、处理、切分、防泄漏	数据来源表 + 特征类别图
Search Strategy	算法原理、复杂度、代码设计	BFS→Bi-BFS→CH 与 UCS→A*→ALT A* 两组对比 + 路线图
Machine Learning	标签、模型、特征、训练、指标	输入输出图 + 结果表 + SHAP
Performance	search results、ML results、complexity、limitations	结果表 + 算法复杂度/取舍图 + limitations

建议分工

每个负责人交付两类内容：report 段落 + PPT 要点。

负责人	章节	需要交付
A	Introduction + final story check	项目概览、系统流程图、demo 截图，并最终检查整份 PPT 叙事是否连贯
B	Problem Definition + CSP	输入/约束/目标表，pre-CSP 与 post-CSP 的 feasibility 解释
C	Dataset	路网、站点、CH index、POI、UrbanEV 占用率数据来源与处理流程
D	Search Strategy + search results + complexity	BFS→Bi-BFS→CH 与 UCS→A*→ALT A* 两组搜索策略，说明 CSP 如何嵌入搜索流程，算法图，搜索结果和复杂度取舍
E	Machine Learning + ML results + limitations	XGBoost 模型、特征、训练切分、防泄漏设计、结果图、ML 指标和 occupancy/ranking 局限

Performance 不建议单独分给第六个人。5 人版本里，search results 和 complexity 由 D 负责，ML results 和 limitations 由 E 负责，最终 summary 由 A 在整合阶段收口。

最终整合人需要统一：

- 术语：统一使用 occupancy rate，不要混用 waiting time，除非明确解释。
- 指标：所有数值必须来自同一份结果文件或文档。
- 图表风格：标题、单位、颜色保持一致。
- 叙事：每章结尾都要连接到下一章。
- 代码路径：只放关键路径，不要把 PPT 变成代码 walkthrough。

最终检查清单

提交前每个章节负责人自查：

- 本节是否回答了“为什么需要这一部分”？
- 是否至少有一张图、表或流程图支撑？
- 是否引用了项目中的真实文件、代码路径或实验结果？
- 是否避免了和其他章节重复？
- 是否说明了局限或设计取舍？
- PPT 中是否每页只有一个核心信息？

整合前全组自查：

- Introduction 是否清楚说明项目价值？
- Problem Definition 是否把约束和目标讲明白？
- Dataset 是否讲清楚时间切分和无泄漏？
- Search Strategy 是否按 BFS→Bidirectional BFS→CH 和 UCS→A*→ALT A* 两组逻辑组织？
- Machine Learning 是否正确说明预测目标是 future occupancy rate？
- Performance 是否覆盖 search results、ML results、complexity 和 limitations？